

Introduction to Programming !

CSE 1110: ICS

Lecture Abdullah Al Jobair

Dept of CSE, UIU





So we have seen that Computer Seems like a very smart machine that can do a lot of Stuff!

But how does it do it?

It Processes and
Executes our orders
(processing)

But How



It doesn't know how to do anything!



It's really dumb!



Unless 'you' make it smart!



But how do you make it smart?

You tell it what to do!

And you do it with **computer programs!**

But again, computers are too dumb to understand you directly!

You gotta speak their language!

So you've got to speak in **computer programming!**



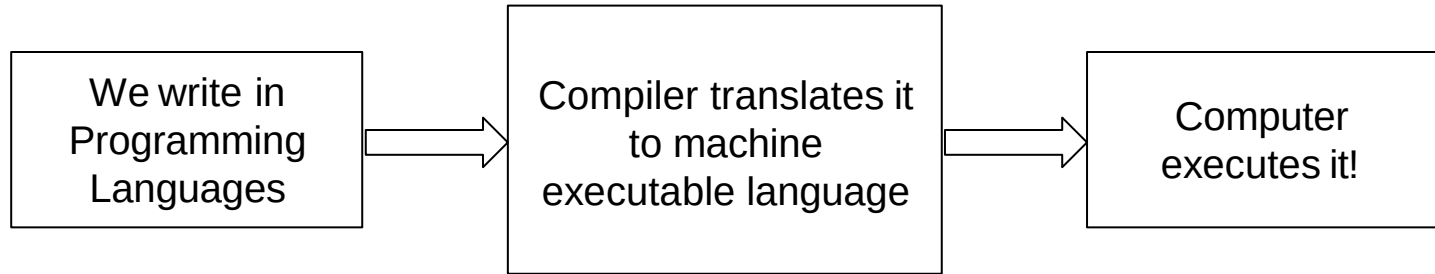
Computer Programs

Computer programming is the process that professionals use to write code that instructs how a computer, application or software program performs. At its most basic, computer programming is a set of instructions to facilitate specific actions.

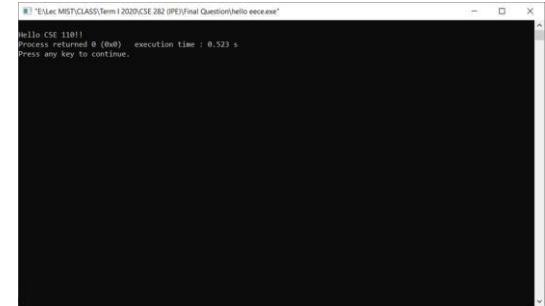
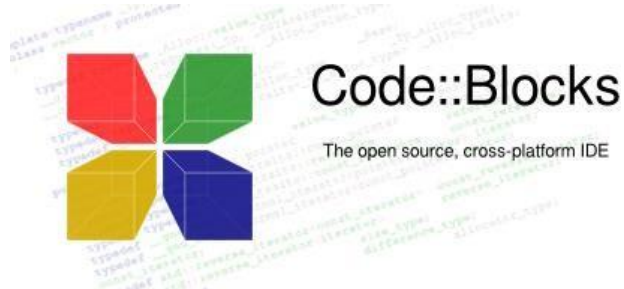
So, it -

- **Tells** the computer **how to do/solve a specific task**
- Gives Detailed **step by step instructions** to computers
- These instructions are **easily understandable** to computers.
- Just Like human languages, there are **many programming languages**, each better suited for a particular type of task.
- Written (mostly) **by humans**.

So how do we make a computer 'smart'?

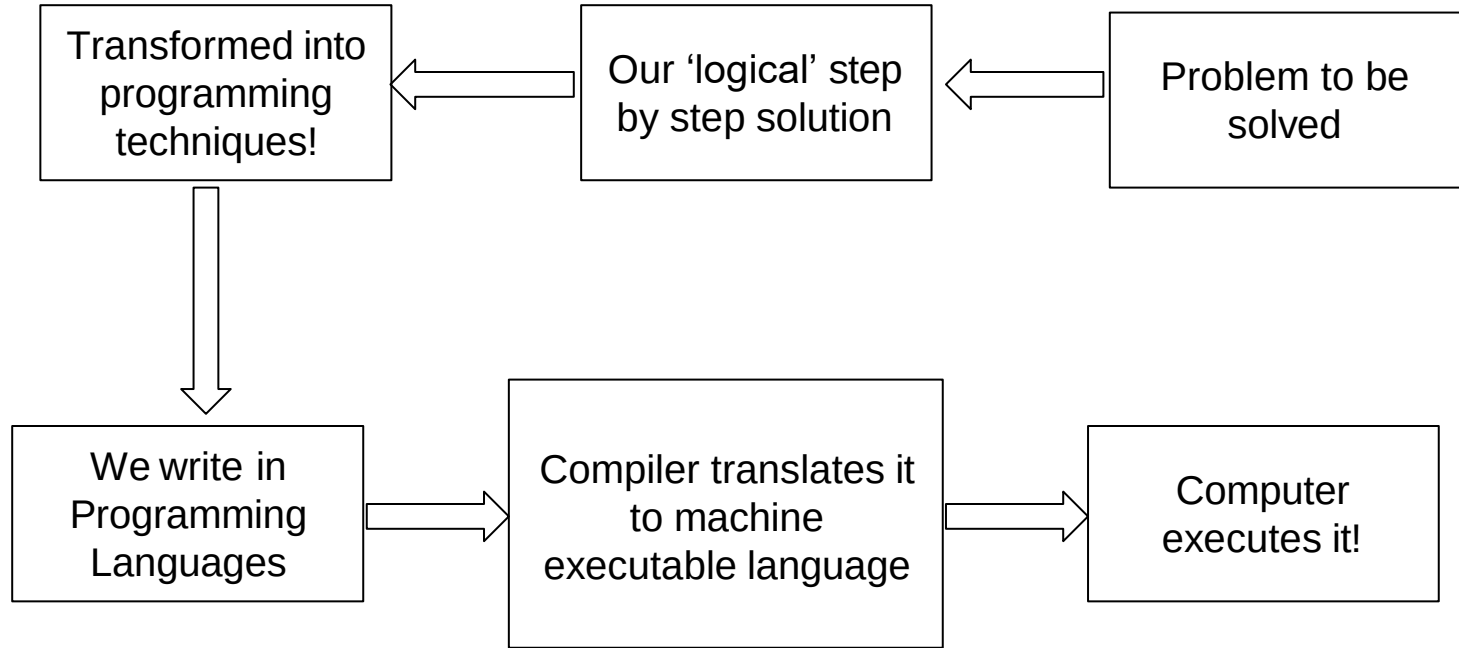


C/C++

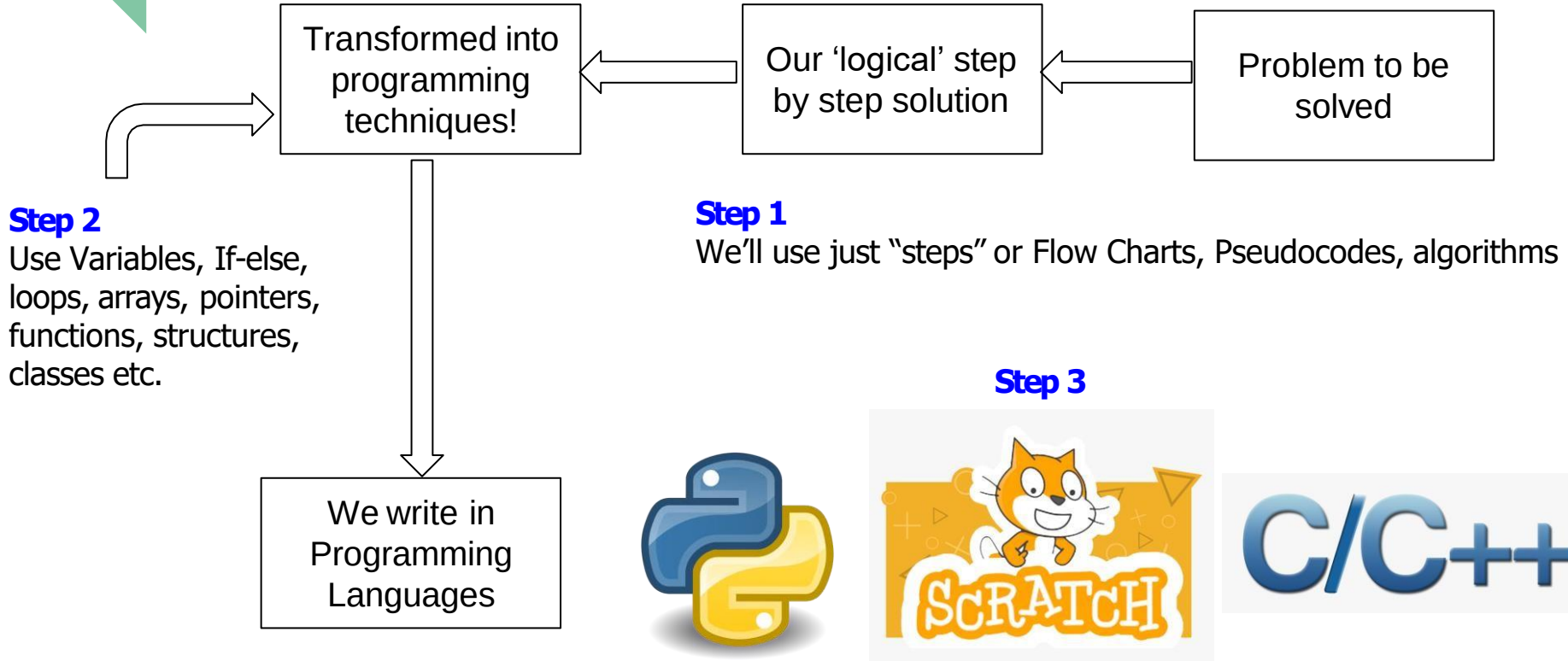


IDE + C/C++ compiler

So how do we make a computer 'smart'?



So how do we make a computer 'smart'?





Some Simple Tasks to solve by own !!!

1. Do you know how to find out your PC's IP?
2. What is an IP address?
3. How do you actually remove a program from your PC?
4. Check which "devices" are installed on your PC.
5. What are device "drivers"?



What we hope to Achieve in CSE 1110

To be able to create our own programs that can make the computer do a WIDE range of things!

A COVID Symptom Detection Tool: [Here](#)

A Library management System: [Here](#)

A Personal Website (maybe)

You **should (hopefully)** be able to write such programs yourself after this course (CSE 115+116).



Let's do it!

- Download the compiler +IDE
- Know the basics of C programming
- Write our first program
- Solve our first problems using computers!
 - Finding a real life problem
 - Breaking it down to "Steps"
 - Transforming it to programming techniques
 - Coding it!



Let's do it!

- Download the compiler + IDE

<http://www.codeblocks.org/downloads>

- For codeblocks installation guide, check:

<https://www.youtube.com/watch?v=GWJqsmiR2I&t=303s>

Let's break down some important concepts in a simple way:

1. IDE (Integrated Development Environment):

An IDE is like a digital toolbox for programmers. It's a software application that provides a set of tools to make writing and running code easier. Ex: CodeBlocks

Here's what it typically includes:

- Code Editor:** This is where you write your code. It often has features like syntax highlighting (color-coding your code), auto-completion (helping you type faster), and error checking.
- Build and Run Tools:** An IDE can help you compile and run your code with just a few clicks. For C programming, it will compile your C code into an executable program and run it.
- Debugging Tools:** IDEs have tools to help you find and fix errors or bugs in your code. You can set breakpoints to stop the program at specific points and inspect variables.
- Project Management:** You can organize your code into projects and manage files more easily within the IDE.

So, in simple terms, an IDE is like a one-stop-shop for writing, building, and debugging your code.

2. Path Environment Variable:

The "path" in computing is like a set of directions that your computer follows to find programs or files. The "Path Environment Variable" is a list of folders or directories that your operating system looks into when you run a command or program without specifying its full path.

For example, when you open a command prompt and type "codeblocks" to run the Code::Blocks IDE, your computer needs to know where to find the Code::Blocks program. It checks the folders listed in the "Path Environment Variable."

Here's a simple analogy: think of the "Path" as a list of bookmarks. When you tell your computer to open a program, it looks at these bookmarks (folders) to find the program's location.

So, when you install Code::Blocks or any other software, it's a good practice to add its installation directory to the "Path Environment Variable." This way, you can run the program from anywhere in your command prompt or terminal without needing to specify the full path.



Example 1- A Simple Conversation

Let's write a program (a chatbot) to have a simple conversation with the user. In time, we'll make more of a 'intelligent' chatbot!

[Check out the solution in Hello_Chatbot.c](#)

Programming

(Standard I/O, Variables, Arithmetic operators)



Standard I/O

- Abbreviated from Standard input output
- input via keyboard
- output via monitor

Structure of C Program

```
#include <stdio.h>
```

Preprocessor Commands

```
int main()
```

Main Function

```
{
```

```
/* my first program in C */
```

Comment

```
printf("Hello! \n");
```

Library Function

```
return 0;
```

End of Program

```
}
```



Preprocessor Commands

- These commands tell the compiler to do preprocessing before doing actual compilation.
- Like **#include <stdio.h>** is a preprocessor command which tells a C compiler to include **stdio.h** file before going to actual compilation.



Functions

- Functions are main building blocks of any C Program.
- Every C Program will have one or more functions and **there is one mandatory function which is called main() function.**
- The C Programming language **provides a set of built-in functions.**
- In the above example printf() is a C built-in function which is used to print anything on the screen.



Comments

- Comments are used to give additional useful information inside a C Program.
- All the comments will be put inside `/*...*/` as given in the example above.
- A comment can span through multiple lines.

```
/* comment for multiple line */
```

```
// single line comment
```



Variable and Variable Declaration

Variable is a **named memory location** that can hold various values.

All variables **must be declared** before they can be used.

When we declare a variable, we tell the compiler what type of variable is being used.

A declaration associates a group of variables with a **specific data type**.



Variables

- Variable is a data name that is used to store a data value.
- A variable may take different values at different times.

Data value	11	c	3	d	1	3	q	0
address	000	001	010	011	100	101	110	111
variable	var1	a	a1	t	t1	b	d	g

How to Declare Variables

To declare a variable, use this general form:

type var-name;

In C, a variable declaration is a statement and it must end in a semicolon (;).

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char ch;
```

```
    int i;
```

```
    float f;
```

```
    double d;
```



Variable Naming Convention

- letters, digits and underscores(_),dollar sign(\$) characters are allowed only
- should not start with digits(0-9)
- upper case and lower case are not identical
- should not be a keyword such as char, int etc.
- white space is not allowed



Variable

Variables consist of **letters and digits, in any order**, except that **the first character must be a letter**.

Both upper-and lowercase letters are permitted, though common usage favors the use of lowercase letters for most types of variables.

Upper- and lowercase letters are **not interchangeable** (i.e., an uppercase letter is not equivalent to the corresponding lowercase letter.)



Variable (cont.)

The underscore character (_) can also be included, and is considered to be a letter.

An underscore is often used in the middle of an variable.

A variable may also begin with an underscore, though this is rarely done in practice.



Variable (cont.)

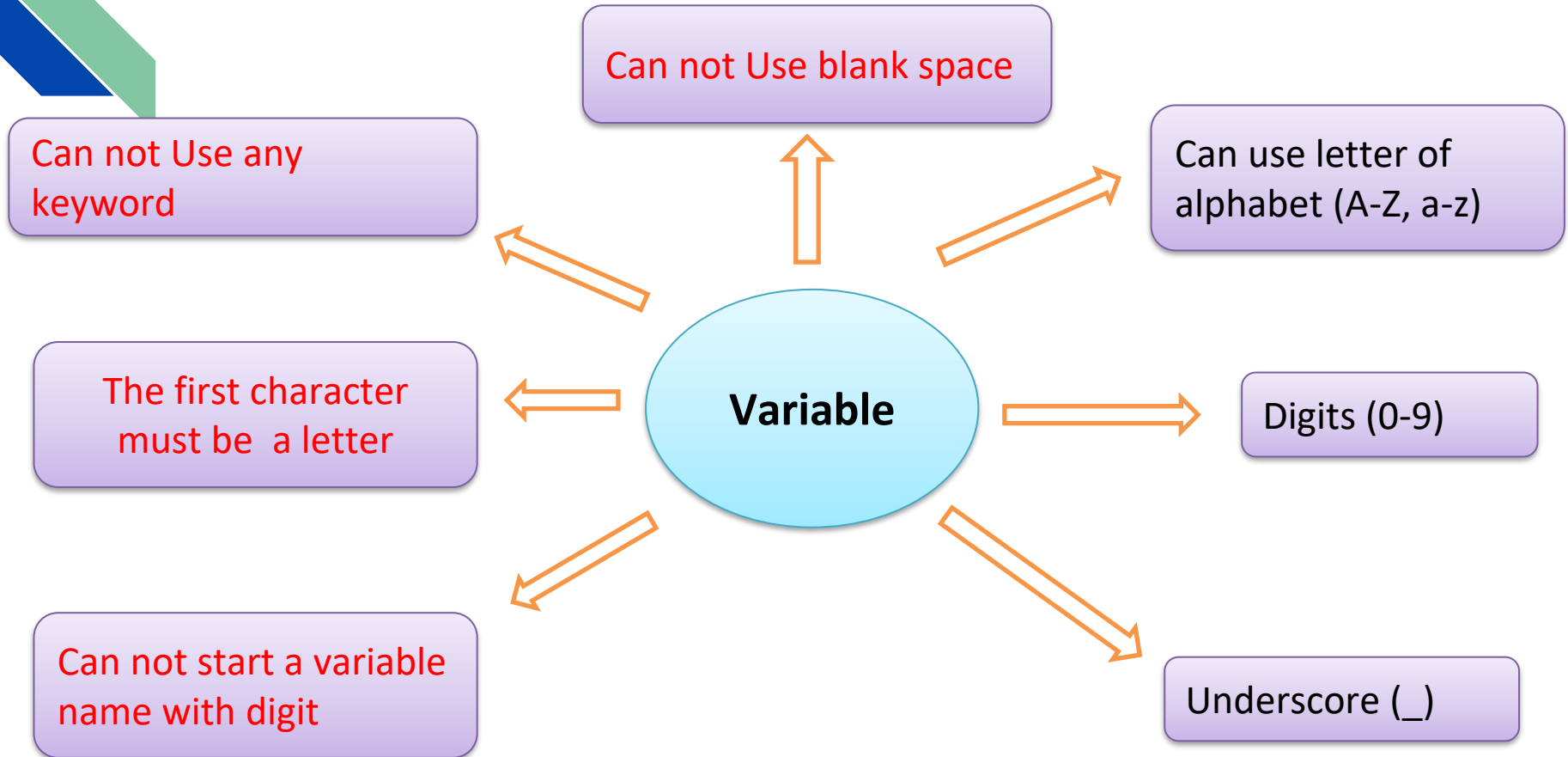
Case-sensitive

COUNT and **count** are not same

As a rule, an identifier should contain enough characters so that its meaning is readily apparent.

On the other hand, an excessive number of characters should be avoided.

Variable (Cont.)



Is it Valid Variable Name?

Apon



1joty



apon



joty-5



apon123



this_is_a_long_name



_sojeb_1



VaReNdRa



Is it Valid Variable Name?

4th



The first character must be letter

"x"



Illegal characters ("")

Order-no



Illegal characters (-)

My variable



Illegal characters (blank space)



Valid & invalid Variable names

Variable name	Valid/invalid
New	valid
_a	valid
a\$	valid
sum?	invalid
first_tag	valid
char	invalid
1st_num	invalid
int_type	valid





Identifiers

Identifiers are names that are given to various program elements, such as variables, functions and arrays.



Keywords

There are certain reserved words, called Keywords, that have standard, predefined meaning in C

Can be used only for their intended purpose

Can't use as identifiers



Keywords

The standard keywords are

auto	extern	sizeof
break	floatn	static
case	for	struct
char	goto	switch

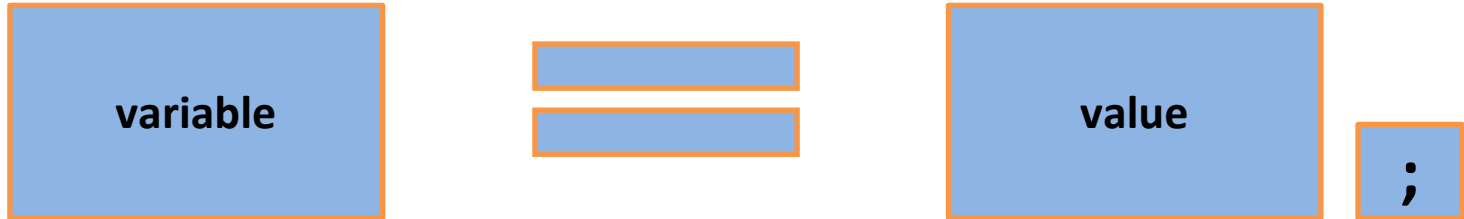
Some compilers may also include some or all of the following keywords.

ada	far	near
asm	fortran	pascal
entry	huge	



Assign value to variable

To assign a value to a variable, put its name to the left of an equal sign (=).
Put the variable you want to give the variable to the right of the equal sign.
It is a statement, so end with a ';'



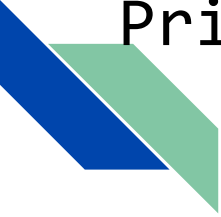


Data Type

Data type is used to define the type of value to be used in program.

Three main types

1. Primary
2. Derived
3. User defined



Primary Data Type

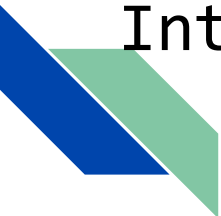
Datatype	Keyword	Size	Examples
character	char	1 byte (8 bits)	'a', '1'
integer	int	4 bytes (32 bits)	1,23,345,-123,-345
Floating point	float	4 bytes (32 bits)	5.5,0.45,-12.32
Double floating point	double	8 bytes (64 bits)	2.22507e-129, -1.79769e+75

Character

char C;

Value of C	Print(c)
C = 'a'	a
C = '4'	4
C = '44'	4
C = 44	,
C = 105	i
C = 125	}

33 !	34 "	35 #	36 \$	37 %	38 &	39 '	40 (	41)	42 *
43 +	44 ,	45 -	46 .	47 /	48 0	49 1	50 2	51 3	52 4
53 5	54 6	55 7	56 8	57 9	58 :	59 ;	60 <	61 =	62 >
63 ?	64 @	65 A	66 B	67 C	68 D	69 E	70 F	71 G	72 H
73 I	74 J	75 K	76 L	77 M	78 N	79 O	80 P	81 Q	82 R
83 S	84 T	85 U	86 V	87 W	88 X	89 Y	90 Z	91 [	92 \
93]	94 ^	95 _	96 `	97 a	98 b	99 c	100 d	101 e	102 f
103 g	104 h	105 i	106 j	107 k	108 l	109 m	110 n	111 o	112 p
113 q	114 r	115 s	116 t	117 u	118 v	119 w	120 x	121 y	122 z
123 {	124	125 }	126 ~	127 �	128 �	129 �	130 �	131 �	132 �
133 �	134 �	135 �	136 �	137 �	138 �	139 �	140 �	141 �	142 �
143 �	144 �	145 �	146 �	147 �	148 �	149 �	150 �	151 �	152 �
153 �	154 �	155 �	156 �	157 �	158 �	159 �	160 �	161 �	162 �
163 �	164 �	165 �	166 �	167 �	168 �	169 �	170 �	171 �	172 �
173 �	174 �	175 �	176 �	177 �	178 �	179 �	180 �	181 �	182 �
183 �	184 �	185 �	186 �	187 �	188 �	189 �	190 �	191 �	192 �
193 �	194 �	195 �	196 �	197 �	198 �	199 �			



Integer

```
int a;
```

a	Print(a)
a = 124	124
a = -123	-123
a = 0.2	0
a = 2e2	200
a = 2e-3	0
a = 2e4	20000



variable

```
int i;  
float f;  
char c;
```

i, f and c are variables

Variables can store data

i can store an integer

f can store a fraction, or floating-point number

c can store the ASCII value of a character

These three are the **primitive data types**



Variable

```
int i = 5;  
float f = 3.14;  
char c = 'G';
```

These variables are initialized with some values
Every variable in a program must be initialized
before using

What does an uninitialized variable
contain?

Note that the character G is enclosed by
single quotes

Basic output: printf

These are format specifiers
and are replaced by the values
of the variables written later

```
printf("Integer: %d\n", i);  
printf("Float: %f\n", f);  
printf("Character: %c\n", c);
```



ment

Workspace

es list

in.c

Start here main.c

```

1  #include<stdio.h>
2  #include<stdlib.h>
3
4  int main()
5  {
6      char ch;
7      int i;
8      float f;
9      double d;
10
11     ch = 'X';
12     i = 1000;
13     f = 100.12345;
14     d = 123.123456789012;
15
16     printf("%c\n", ch);
17     printf("%d\n", i);
18     printf("%f\n", f);
19     printf("%lf", d);
20
21     return 0;
22 }
23
24

```

"E:\C Code\Teach Yourself C\Chapter 01 - Fundamentals\variables\main.exe"

```

X
1000
100.123451
123.123457
Process returned 0 (0x0)   execution time : 0.015 s
Press any key to continue.

```



Arithmetic operators

Operator	Meaning
+	Addition or unary plus
−	Subtraction or unary minus
*	Multiplication
/	Division
%	Remainder

$a - b$

$a * b$

$a + b$

$a \% b$

a / b

Basic arithmetic operations

`p = 3 + 5;` The five basic arithmetic operations are addition (+),
`q = 9 - 7;` subtraction (-), multiplication (*), division (/) and
`r = 2 * 8;` remainder (%)
`s = 12 / 6;` There is no power operator in C
`t = 17 % 5;` Remainder operation is not available for float variables
Here, each variable is assigned the result of the
corresponding operation



Precedence of arithmetic operators

`x = 6 * 4 % 3 + 12 - 15 / (2 + 1) + 9;`

Multiplication, division and remainder has the highest precedence

Addition and subtraction has the lowest precedence

Operations of the same precedence are left-to-right associative

Almost like BODMAS



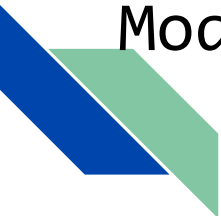
Arithmetic operators

- let a and b are integers and $a = 14$, $b=4$
- $a + b = 14 + 4 = 18$
- $a - b = 14 - 4 = 10$
- $a * b = 14 * 4 = 56$
- $a / b = 14 / 4 = 3$
- $a \% b = 14 \% 4 = 2$

Division operator

- if the result of division is supposed to float, then at least one of the operands should be float
- `int / int` $\rightarrow$ `int`
- `int / float` $\rightarrow$ `float`
- `float / int` $\rightarrow$ `float`
- `float / float` $\rightarrow$ `float`

<code>4 / 5</code>	<code>0</code>
<code>4.0 / 5</code>	<code>0.8</code>
<code>4 / 5.0</code>	<code>0.8</code>
<code>4.0 / 5.0</code>	<code>0.8</code>



Modulo operator

- both operand should be integer
- Sign of result is always the sign of the first operand
- $5 \% 3 = 2$
- $3.4 \% 3 = \textit{not defined}$
- $14 \% 3.3 = \textit{not defined}$
- $15 \% 3 = 0$
- $-14 \% 3 = -2$
- $-14 \% -3 = -2$
- $14 \% -3 = 2$



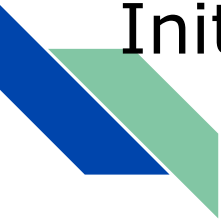
Example

Write a C program where you will declare two integer variables, initialize them by values of your choice, and perform the basic arithmetic operations on them.



exercise

Write a C program where you will declare four integer variables (say a, b, c and d), initialize them by values of your choice, and calculate $a * b + (a - c) / d + b$.



Initializing variables

By assigning a constant value

By using an expression

From the user



Input Numbers From Keyboard

There are several methods
The easiest is – **scanf()** function.

[Input from Keyboard](#)

Basic input: scanf

The same format specifiers
are used in scanf, but additional
texts must be removed

```
scanf("%d", i);  
scanf("%f", f);  
scanf("%c", c);
```



Build Debug wxSmith Tools Plugins Settings Help

main() : int

Start here *comment.c x

```
1  /* This program will take two input
2  | from the user and print their sum.*/
3
4  #include <stdio.h> //Library file access
5
6  int main()          //Main Function heading
7  {
8      int num1, num2; //Variable declare
9
10     printf("Enter first number: ");
11     scanf("%d", &num1);           //Input 1st number
12     printf("Enter second number: ");
13     scanf("%d", &num2);           //Input 2nd number
14
15     printf("Sum = %d\n\n", num1+num2); //Print output
16
17     return 0;
18 }
19
```



Example

Write a C program where you will declare two integer variables, input them using scanf, and perform the basic arithmetic operations on them.



Example 2 - Age Calculator

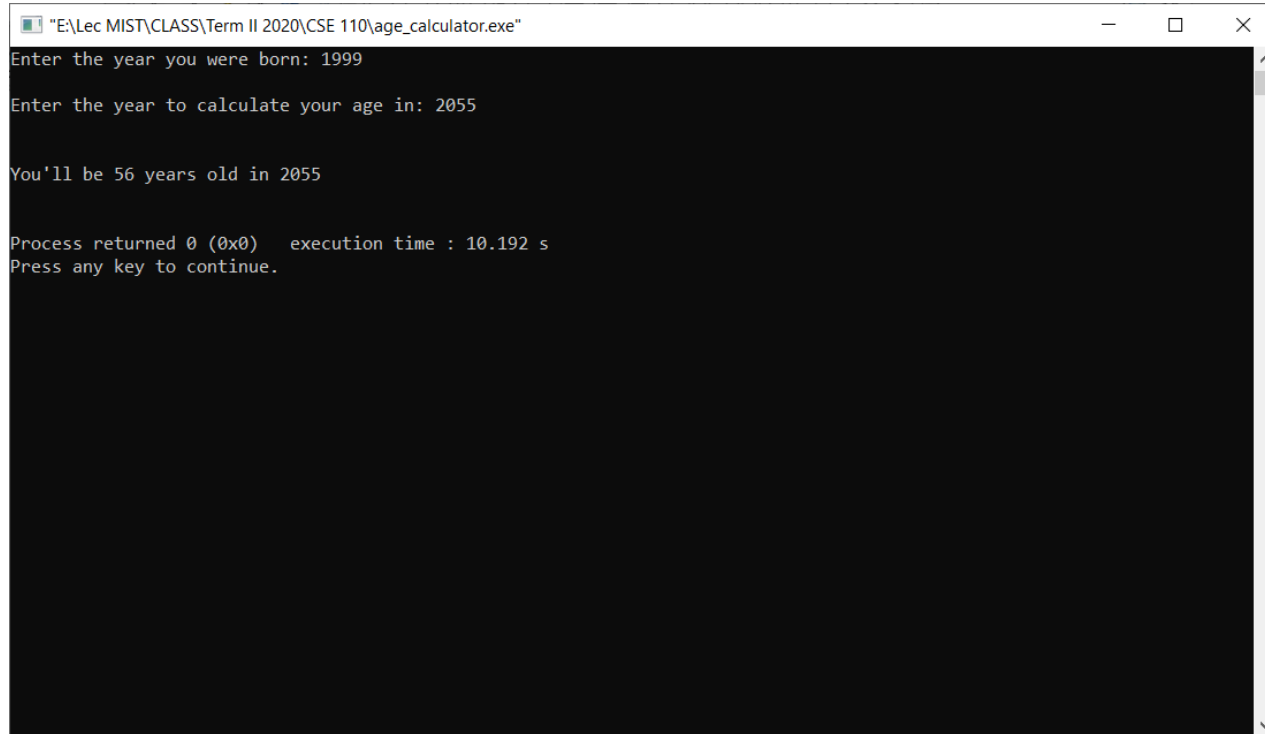
Make a Age Calculator that takes the birth year as input from the user and tells them how old they'll be in a particular year!

Input (To be taken from users)	Output
Enter The year you were born: 1999 Enter the year to calculate your age in: 2055	You'll be 56 years old in 2055.

[Check out the solution in age_calculator.c](#)

Example 2 - Age Calculator

Here's what the output should look like:



```
"E:\Lec MIST\CLASS\Term II 2020\CSE 110\age_calculator.exe"
Enter the year you were born: 1999
Enter the year to calculate your age in: 2055

You'll be 56 years old in 2055

Process returned 0 (0x0)   execution time : 10.192 s
Press any key to continue.
```



Example 2 - Age Calculator

Let's try out the steps for writing programs!

Step 1	Step 2
Ask the user when he/she was born	printf()
Store their answer (in birth_year)	scanf() + use an int variable
Ask and store the year he/she wants to calculate the age in (in calc_year)	printf() + scanf() + use an int variable
Do calc_year - birth_year to find age	Arithmetic operation (-)
Show them the result!	printf()



Escape Sequences

There are certain characters in C when they are preceded by a backslash (\) they will have special meaning.

They are used to represent like newline (\n) or tab (\t).



Escape Sequences for printf()

Escape Sequence	What it does
<code>\n</code>	Prints a new line. More specifically, moves the cursor to a new line.
<code>\t</code>	Moves the cursor one tab to the right.
<code>\r</code>	Moves the cursor to the beginning of the current line. Then whatever you write after <code>\r</code> will overwrite the previous characters. This is called carriage return.
<code>\b</code>	moves the cursor back one space. Then whatever you write overwrites what you had in that space. However, it can't go back to a previous line.
<code>\a</code>	Doesn't move the cursor. Just makes a “beep” or windows “alert” sound for that line.
<code>\</code>	Put <code>\</code> before any of the above escape sequences from above if you want to print the escape sequence itself rather than to move the cursor.
<code>%%</code>	However, if you want to print <code>%</code> itself just put another <code>%</code> before that. Putting <code>\</code> before <code>%</code> won't work.

Note: Escape characters take effect within that `printf()` (that line) only.

Escape Sequences

Escape sequence	Meaning
\\	\ character
\'	' character
\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits



Defining Constants

There are two simple ways in C to define constants:

- Using ***#define*** preprocessor.
- Using ***const*** keyword.

Using **#define** preprocessor

Following is the form to use #define preprocessor to define a constant:

Syntax: `#define identifier value` at the beginning of a program.

- Name of the constant
- Can be any valid identifier name

- Constant value
- Can be character/numeric/string



Using **#define** preprocessor

```
#define identifier value
```

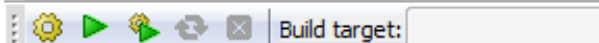
```
#define MAX 10
```

```
#define PI 3.142
```

```
#define NEWLINE '\n'
```

```
#define ANSWER 'Y'
```

```
#define CR "ANABI"
```



Management

Projects Symbols Resol

Workspace

Open files list

comment.c

main.c

Start here

*comment.c

main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int a, b, sum;
7
8      a = 10;
9      b = 10;
10
11     sum = a+b;
12
13     printf("The summation is %d\n\n", sum);
14
15     return 0;
16 }
17
```



What we learnt

1. Writing a simple program in c.
2. What header files are (stdio.h in our program)
3. How to use printf().
4. How to use scanf().
5. Some basic idea on how to store an input and show it.
6. Operators
7. Making a little chatbot program in C!

